

ETA — Project Status and Field-Test Readiness

Oct 10, 2026 · @Crumble

Stakeholder brief

The software for all ten build stages exists and passes its automated checks, but it has only ever run against simulated buses. No real phone, bus, route, SMS or production server has been used yet.

- **What it is.** A phone on the bus sends GPS every 5 seconds. A server snaps each fix to the route, works out a per-stop arrival time, and pushes it live to students' browsers. An admin console lets the Transport Department manage buses and send alerts.
- **What is proven.** Code review against the design found the core tracking, live map, ETA, alerts and admin flows genuinely implemented, not stubbed. 431 automated tests pass. Earlier simulator runs recorded 30 buses for an hour with no lost or duplicated fixes, and a 4-second p95 delay from GPS fix to screen.
- **What is not proven.** ETA accuracy on a real road (target: under 90 seconds error at 10 minutes out), behaviour on a real phone over a mobile network, real SMS, iPhone alerts, and the production deployment.
- **What blocks progress.** Four requests to people, none of them sent yet: SMS sender registration (1 to 2 weeks), the student roster, real ridership numbers, and one cooperative driver on one route for about a week.
- **Next step.** A field test with a driver's phone on one real route. It needs two fixes to the test setup first, described under *Driver GPS field test*.
- **Cost.** About Rs 67,000 a year at pilot and Rs 95,000 at full fleet, against the Rs 37,700 in the original deck. Per student it is still about Rs 20 a year.

What this review ran

The static checks pass on this machine; the live stack could not be started here because Docker is not installed and there is no `.env` file.

Check	Result on 2026-10-10, commit <code>ba270de</code>
<code>pnpm install</code>	Clean, 43 s
<code>pnpm typecheck</code>	14 of 14 packages pass
<code>pnpm lint</code>	14 of 14 packages pass
<code>pnpm test</code>	431 passed, 0 failed, 138 skipped. The skipped suites need a running Redis or Postgres 15
<code>pnpm build</code>	Driver app builds (254 kB, 80 kB gzipped). Web app build stops at <code>NEXT_PUBLIC_SUPABASE_URL is not set</code> , a missing-configuration stop, not a code error
Full stack (<code>pnpm dev</code>)	Not run: no Docker, no <code>.env</code> , no prepared routing or map data on this machine
Code audit	Three Sonnet agents read the source against ARCHITECTURE sections 3, 5, 6, 7 and 10. I re-read the findings that affect the field test

Every performance number in this document (latency, load, zero-loss ingest) comes from the repository's own benchmark notes, recorded on another machine in September. This review did not reproduce them.

Stage-by-stage status

One stage of ten is fully closed. The other nine have their code built and are waiting on a real-world step, not on more programming.

Stage	Code	Still open
0 Foundations	Built	SMS registration and roster request drafted in <code>docs/OUTREACH.md</code> , not sent
4 Identity and roster	Built	A claim with a real SMS (blocked on SMS registration)
1 Geo core and route capture	Built, 100% line coverage on the maths package	One real route surveyed and published

Stage	Code	Still open
2 Ingestion	Built	Airplane-mode test on a physical phone over mobile data
3 Live delivery	Complete	Nothing
5 Search and ETA	Built	ETA accuracy over 10 or more real trips; a stopwatch-timed walk
7 Admin console	Built	Usability session with a Transport Department staff member
6 Notifications	Built ahead of its gate	iPhone push, alert delay on a real phone, real SMS
8 Learning, observability, load	Built, measured on the simulator	Three or more dead zones learned on real routes
9 Production and hardware	Built, proven locally	Production accounts, the pilot, a physical GT06 tracker

Stages are listed in build order. Stage 6 was built before Stage 5's accuracy gate was met, which the build plan says not to do; the README says leave-now alerts are to be switched on only once accuracy is proven.

The tracker file says the repository was built on a Mac. Nothing in it has been run on this Windows machine before today.

Can it do what ARCHITECTURE.md describes?

Yes for the mechanisms, on the evidence of reading the code: every section is implemented in real code with no stubs found. Not yet for the outcomes, because the accuracy and latency targets are only measured on a simulator.

Architecture section	Verdict	What the audit found
2.4 Redis live, Postgres record	Implemented	Ingest writes only to a Redis stream; the persister batches into Postgres separately

Architecture section	Verdict	What the audit found
3 Ingest path	Implemented	Signed requests, 5-minute clock window, replay cache, 120 requests a minute per device
5.2 to 5.4 Snap, progress, stop crossing	Implemented	Thresholds match the document, including the 4-ping re-snap and the skipped-stop rule
5.5 ETA	Partial	The blend, speed limits and fallback ladder exist. Stop dwell time is a fixed 30 s, never learned. Learned speeds depend on a nightly database job that only exists where <code>pg_cron</code> is available
5.6 Leave-now	Implemented	Fires once through a database compare-and-set, only for a live bus with a fresh fix. The buffer does not widen on low confidence as the document says
5.7 Presence and dead zones	Implemented	Thresholds are 3 and 9 times the reported cadence
5.7 Backfill rules	Partial	Late pings never overwrite live state or notify. Late pings that arrive after a newer fix produce no stop history, which the document promises
6 Notifications	Implemented, three gaps	Tiers, de-duplication in the database, filters, push then SMS, kill switch all present. Gaps are listed in the next section
7 Live stream	Implemented	Login token, 15 s heartbeat, resume, 3-connection limit with self-expiring keys. Replay is sent before the ready frame, the reverse of the documented order

Architecture section	Verdict	What the audit found
10 Security	Implemented	Row-level security on every table; tracker secrets encrypted, not hashed; admin role read from the database on each request
12 Observability	Not audited	Code is present; no Grafana account exists to run it against

Not audited at all: the web interface screens, the GT06 hardware adapter, and the production deployment configuration.

Gaps and risks

Five things stand between today and a phone on a bus; the first is a documentation error that would stop the test at the first screen.

Blocks the field test

- 1. The phone must load the driver app over HTTPS.** The app signs requests with `crypto.subtle` and uses geolocation and the screen wake lock. Browsers provide all three only on HTTPS or `localhost`. The README and `.env.example` tell you to point the phone at `http://<laptop IP>`, which will not work. Nothing in the repository sets up HTTPS for local use, and the app does not detect the problem.
- 2. No real route exists.** The database only gets eight simulator routes. A trip on a road with no published route shows as off-route with no ETA. The first drive has to be a survey.
- 3. This machine cannot run the stack yet.** No Docker, no `.env`, no routing or map data. The setup scripts are bash and were written on macOS; they are untested on Windows.
- 4. The gateway accepts one driver-app address only.** `DRIVER_ORIGIN` must equal the exact address the phone loads, or every request is refused and the pairing link points to the wrong place.
- 5. A slow phone clock hides the bus.** A fix timestamped more than 30 s behind the server is treated as a late upload (`apps/gateway/src/routes/ingest/index.ts:105`), and late uploads never move the live map. The phone must be on automatic network time.

Defects found by reading the code

None of these was reproduced by running the system. Each is a code path the audit read end to end unless marked suspected.

Defect	Effect	Where
<i>Follow</i> does not clear a <i>Not today</i> mute	A student who mutes a bus and then follows it gets no leave-now push that day	<code>packages/notify/src/tiers.ts:94</code>
Engine accepts console SMS in production	A critical SMS is logged, not sent, yet recorded as texted	<code>apps/engine/src/main.ts:132</code>
Lost trip state marks earlier stops skipped	After a Redis flush mid-trip, students may get a burst of "passed stop" alerts	<code>packages/geo/src/crossings.ts:127</code>
Scheduled announcements skip the count re-check	The audience can change between scheduling and sending without a new confirmation	<code>apps/engine/src/workers/announcements.ts:25</code>
Per-bus mute also silences critical alerts	A muted bus going out of commission sends no push or SMS (suspected: intent unclear)	<code>packages/notify/src/tiers.ts:94</code>

Defect	Effect	Where
Live speed average includes time stopped	ETAs may jump up after each stop (suspected impact)	<code>packages/geo/src/trip.ts:158</code>
"Delay over 15 minutes" alert is not wired	The tier table lists it; no code emits it	<code>apps/engine/src/lib/notify-plans.ts</code>
Gateway trusts forwarded-address headers	Per-address login limits can be bypassed if exposed without a proxy	<code>apps/gateway/src/app.ts:82</code>

Waiting on people

- SMS sender and template registration with MSG91: drafted, not submitted, 1 to 2 weeks once sent.
- Roster file, ridership figures and the cohort definition from the Transport Department: one drafted letter, not sent.
- A driver and one route for about a week: not arranged. This is the gate for ETA accuracy and for dead-zone learning.
- Open decision for you: publish route data under the ODbL licence (ADR-0010 recommends yes).

Driver GPS field test

The test runs in four steps: get the stack running, prove the phone at a desk, survey one route, then drive tracked trips on it. Use an Android phone with Chrome; the driver app has not been tried on an iPhone.

1. Get the stack running on this machine

1. Install Docker Desktop with the WSL 2 backend. Work inside the existing Ubuntu WSL distribution, where the bash scripts, `osmium-tool` and `python3` are available (`sudo apt-get install osmium-tool`). Clone the repository into the WSL file system, not

`C:\New folder`.

2. `pnpm install`, then `cp .env.example .env`.
3. `infra/osrm/prepare.sh` then `infra/tiles/prepare.sh`. One time, about 2 minutes and a 99 MB download.
4. `pnpm dev`. It checks `.env`, starts Supabase, Redis, both routing engines, the map tile server and all four apps, then smoke-tests them. Fill the two empty Supabase keys in `.env` from `npmx supabase status --workdir infra`.
5. Open `http://localhost:3000/claim`, claim roll `TDADMIN01` (the one-time code prints in the gateway log), then make it an admin with the SQL line in `infra/supabase/seed.sql`.
6. Run `pnpm sim seed` and `pnpm sim run --buses 3 --minutes 5`. Three buses moving on the map confirms the whole pipeline before any phone is involved.

The test area must lie inside the prepared map: longitude 78.15 to 78.80, latitude 17.15 to 17.70 (greater Hyderabad).

2. Prove the phone at a desk

Connect the phone by USB with debugging on, then run `adb reverse tcp:5173 tcp:5173` and `adb reverse tcp:4000 tcp:4000`. The phone now reaches the laptop as `localhost`, which browsers treat as secure, so no configuration changes.

1. `pnpm tracker provision --bus 14` prints a pairing link once. Open it in Chrome on the phone.
2. Pick a route, press **START TRIP**, and watch "N pings sent" climb.
3. Confirm Bus 14 appears on the student map on the laptop.

3. Put the phone on the road

The phone needs public HTTPS addresses for both the driver app and the gateway, and the laptop must stay on and online for the whole drive.

1. Open two HTTPS tunnels to the laptop, one to port 4000 and one to port 5173 (for example `cloudflared tunnel --url http://localhost:4000`).
2. In `.env`, set `VITE_GATEWAY_URL` to the gateway tunnel address and `DRIVER_ORIGIN` to the driver-app tunnel address. Restart the gateway.
3. Build the driver app with `pnpm --filter @busmitra/driver build` and serve `apps/driver/dist` on port 5173 with any static file server. Vite's own servers reject tunnel hostnames by default.

4. Provision again so the pairing link carries the new address, and open it on the phone.

Free quick tunnels get a new address on every restart, which means repeating steps 2 to 4. A named tunnel on a domain you own avoids that.

4. Survey, then track

1. **Survey drive.** In the driver app choose the **Route survey** tab, name the route, drive it once end to end, then upload.
2. **Publish.** In the console under Admin, Routes: match the trace, drag any wrong sections, drop and name the stops, publish.
3. **Tracked trips.** On each later run pick that route and press **START TRIP** before moving, **END TRIP** at the last stop. Note the real arrival time at two or three stops by hand as a cross-check.
4. **Airplane-mode test.** Mid-trip, switch the phone to airplane mode for 3 minutes, then back.
5. After each day run `pnpm sim eta-report --since <start time>`.

Phone rules for every trip: mounted, charging, screen on, automatic date and time, battery saver off, location set to "allow while using".

What passes

Check	Pass
Fix reaches the map	p95 under 6 s
Signal lost while moving	Amber at about 15 s, red at about 45 s, removed after 10 min
Bus parked at a stop	Stays green
Airplane mode for 3 min	Every buffered fix arrives with its original time, marked late; the live marker never jumps backwards
Surveyed route	Matched line follows the road driven; stops in the right order
ETA accuracy	Mean error under 90 s at 10 minutes out, over 10 or more trips

Check	Pass
Phone call or notification during a trip	Tracking resumes when the app is back on screen

When a bus does not appear, `vault/runbooks/bus-not-appearing.md` walks the path from phone to map. The driver's one-page sheet is `docs/handover/driver-sheet.md` ; its Telugu and Hindi text still needs a native speaker's check.